MemOS-Plus: User-Centric Memory Management for Language Models

Qingzhi Chen, Jiawei Song, Yixuan Wang, Kaidi Fu, Siqi Zhu, Xuan He
Department of Computer Science
University of Illinois Urbana-Champaign
{qingzhi, jiaweis7, yixuan41, kaidifu2, siqizhu4, xuanhe4}@illinois.edu

Abstract

As large language models (LLMs) evolve toward more capable agents, effective memory management is crucial for long-term reasoning and personalized interaction. While the MemoryOS framework (Kang et al., 2025) provides a foundational hierarchical architecture, it faces limitations in retrieval transparency, storage efficiency, and the handling of diverse information types. In this paper, we propose MemOS-Plus, an enhanced framework designed to address these gaps. We augment the original architecture by introducing explainable retrieval mechanisms, content-aware memory routing, graph-based context associations, and semantic compression. Experiments show that our approach not only improves the system's token efficiency but also enhances user trust through interpretable reasoning, offering a more robust solution for long-horizon human-LLM interactions. Our code is available at: https://github.com/hexuan21/course_proj_MemOS-Plus

1 Introduction

As large language models (LLMs) continue to advance in their fundamental capabilities, there is growing interest in enabling them to sustain longer contextual interactions and perform increasingly complex tasks, such as functioning as autonomous agents. However, the fixed context window of current LLMs remains a major bottleneck, restricting their ability to recall and utilize important information from past interactions over long horizons. Consequently, designing an effective memory system—one that maintains persistent attention to key information and supports efficient retrieval—has become a crucial research direction toward achieving genuine long-term learning and personalized interaction.

Existing approaches, such as traditional Retrieval-Augmented Generation (RAG), typically function as static storage, fetching information based solely on semantic similarity without considering the dynamic nature of human-AI interaction. More advanced hierarchical frameworks, such as MemoryOS, have attempted to address this by mimicking human memory structures: Short-Term Memory (STM) for timely conversations, MidTerm Memory (MTM) for recurring topic summaries, and Long-term Personal Memory (LPM) for user or agent preferences (Kang et al., 2025). While MemoryOS provides a solid architectural foundation, it faces several limitations: it often operates as a "black box" lacking retrieval transparency; it mixes user-specific preferences with general factual knowledge, leading to long-term memory pollution; and it relies on simple list-based structures that miss complex associative links between dialogue turns.

In this work, we propose MemOS-Plus, a user-centric memory management framework designed to overcome these limitations. Specifically, we extend the capabilities of the baseline MemoryOS framework through four key contributions:

Explainable Retrieval: We introduce an interpretation layer that enables the system to generate natural language explanations for its retrieval decisions. By explicitly stating which memories were used to generate an answer, we enhance the transparency and trustworthiness of the agent.

Content-Aware Memory Routing: To address the issue of memory pollution, we design a routing mechanism that distinguishes between *User Profile* (personal preferences) and *Fact Knowledge*. This ensures that personal data and general facts are stored in distinct pathways, facilitating more accurate retrieval for personalized tasks.

Graph-Based Mid-Term Memory: Moving beyond simple list-based buffers, we implement a graph-based structure for mid-term memory. By utilizing cosine similarity and creating edges for every pairs of items with similarity exceeding the threshold, our system can capture richer contextual

associations and retrieve information more effectively.

Semantic Memory Compression: To reduce the cost of memory, we integrate **LLM-Lingua-v2** (Pan et al., 2024) to compress memory records. This allows our system to retain essential information while significantly reducing token usage and storage overhead by up to 60% in the best case.

By integrating these features, MemOS-Plus aims to establish a more scalable, interpretable, and cost-effective framework for long-term memory in agentic LLMs.

2 Related Work

Existing approaches to LLM memory can be broadly categorized into Retrieval-Augmented and Vector Memory, hierarchical architectures, and learning-based controllers. Our work builds upon these foundations but specifically targets the gaps in interpretability and structural efficiency.

2.1 Retrieval-Augmented and Vector Memory

Early practical systems typically implemented conversation memory by logging past turns into an external store (often a vector database) and performing similarity search at inference time. Frameworks such as LangChain popularized plug-and-play "memory modules" (e.g., buffer, windowed buffer, and summary buffers), enabling basic persistence across tasks. While widely adopted, these designs are often unstructured and rely on heuristic retrieval. They treat memory as a flat list of segments, which can accumulate noise and degrade retrieval relevance under tight token budgets (LangChain, 2025).

2.2 Hierarchical Memory Architectures

To address the limitations of flat storage, recent research has shifted toward hierarchical memory systems inspired by human cognitive processes. **MemGPT** introduced a virtual context management system that distinguishes between main context (RAM) and external context (Disk), allowing LLMs to autonomously swap information (Packer et al., 2023). Similarly, **MemoryOS**—the baseline for our work—proposes a multi-level structure comprising short-term memory, mid-term memory and long-term memory. While these hierarchical frameworks improve information organization, they often operate as "black boxes," lacking mechanisms to explain why specific memories were retrieved or how to efficiently distinguish between user-specific profiles and general facts.

A-MEM (Xu et al., 2025) structures experiences as interlinked notes (inspired by Zettelkasten), shifting focus from flat similarity search to a graph of evolving, semantically related memories. However, A-MEM relies heavily on LLM reasoning to explicitly generate links, which incurs high latency. Our work adopts a similar associative philosophy for mid-term memory but proposes a lightweight mechanism that utilizes cosine similarity to autonomously construct associative graphs, ensuring efficient context tracking without the overhead of LLM-based link generation.

2.3 Learning-based Memory Controllers

A parallel line of research explores training specific controllers to manage memory operations. **MEM1** frames long-horizon interaction as a reinforcement-learning problem, consolidating observations into a single compact internal state (Zhou et al., 2025). Similarly, **Memory-R1** trains a dedicated memory manager via outcome-driven reinforcement learning to optimize operations like ADD and DELETE (Yan et al., 2025). While these learning-based methods show promise, they require expensive training pipelines and often lack transparency. In contrast, our approach (MemOS-Plus) focuses on an architectural solution—using explicit routing and interpretation layers—to achieve effective memory management without the high cost of RL training.

2.4 Other techniques towards Memory

Compression As interaction history grows, token efficiency becomes critical. Techniques like **LLM-Lingua** explore prompt compression to reduce the computational cost of processing long contexts without losing essential semantic information (Pan et al., 2024). We incorporate such compression techniques directly into the memory pipeline to maximize information density within the limited context window, rather than merely optimizing static storage.

3 Methods

To address the challenges of long-term consistency , personalization and better memory token efficiency in LLM, we build our system, **MemOS-Plus**, upon the foundational architecture of the *MemoryOS* framework (Kang et al., 2025), as shown in Fig1. In this section, we first review the hierarchical structure of the baseline MemoryOS and then detail the

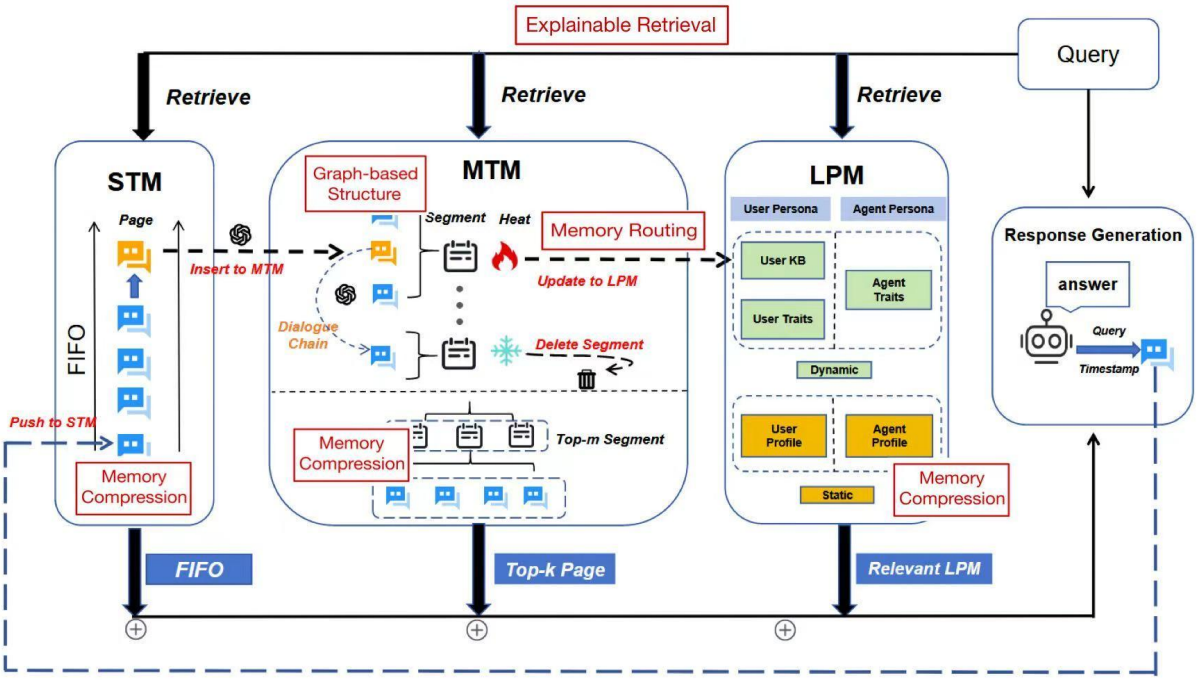

Figure 1: **The Architecture of MemOS-Plus.** The framework extends the original MemoryOS baseline with four key enhancements, highlighted in red boxes: (1) An **Explainable Retrieval** module positioned before response generation to provide transparency; (2) **Semantic Compression** integrated across STM, MTM, and LTM layers to optimize token efficiency; (3) A **Graph-Based Structure** in Mid-Term Memory (MTM) to improve context associativity; and (4) **Content-Aware Memory Routing** between MTM and LTM to effectively segregate user profiles from factual knowledge.

four enhancements we introduce to improve interpretability, efficiency, and structural organization.

3.1 The MemoryOS Baseline Architecture

MemoryOS draws inspiration from the memory management principles of traditional operating systems to manage information across different temporal scales. It organizes memory into a three-tier hierarchy designed to balance retrieval speed with storage capacity:

Short-Term Memory (STM): STM serves as the immediate working context for the agent. It stores the most recent dialogue turns (typically structured as query-response pairs) in a First-In-First-Out (FIFO) buffer. This layer provides the highest fidelity information for immediate reasoning but is strictly limited by the LLM's context window size.

Mid-Term Memory (MTM): When information overflows from STM, it is not immediately discarded but consolidated into MTM. In the original MemoryOS, MTM is typically implemented as a list of summarized dialogue segments or "pages" grouped by topic. This layer acts as a bridge, retaining context that is no longer in the immediate window but is still relevant to the current session.

Long-Term Memory (LTM): For persistent retention across sessions, salient information is promoted to LTM. This layer functions as a large-scale external database (often a vector store) holding two distinct types of data: *User Profile* (preferences, traits) and *Factual Knowledge*. Retrieval from LTM is semantic, relying on embedding similarity to fetch relevant history when needed.

While this hierarchy provides a solid foundation, the original implementation treats the retrieval process as a "black box," often struggles with efficient context linking in MTM, and accumulates noise in LTM over time.

3.2 MemOS-Plus: Enhanced Features

We propose **MemOS-Plus**, which extends the baseline architecture with four key features designed to make the memory system more user-centric, transparent, and token-efficient.

3.2.1 Explainable Retrieval Layer

A critical limitation of standard RAG and MemoryOS is the lack of transparency; users rarely know *why* a specific piece of information was used. To ad-

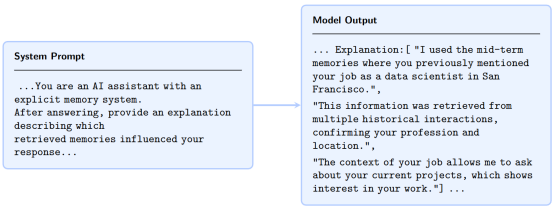

Figure 2: Example output of Explainable Retrieval Layer

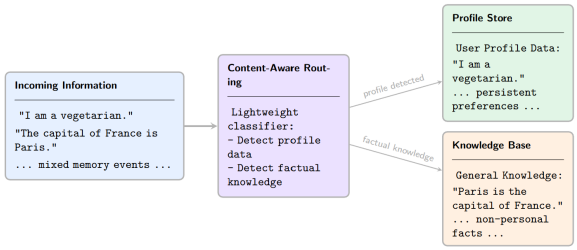

Figure 3: Example of Content-Aware Memory Routing

dress this, we introduce an **Explainable Retrieval** mechanism. Instead of simply feeding retrieved context into the generator, our system includes an interpretation step. When the memory module retrieves segments from MTM or LTM, the agent generates a natural language explanation (e.g., *"I recalled your preference for Python from our conversation last week..."*) alongside the final answer. This feature enhances user trust by making the reasoning process visible and verifiable. An example output of Explainable Retrieval Layer is provided in Fig3

3.2.2 Content-Aware Memory Routing

The original MemoryOS often mixes different types of long-term data, leading to "memory pollution" where trivial facts obscure important user preferences. We implement a Content-Aware Routing mechanism that classifies incoming information before storage. Using a lightweight classifier, the system distinguishes between: (1) **User Profile Data:** High-priority personal information (e.g., "I am a vegetarian") is routed to a dedicated Profile Store. (2) **General Factual Knowledge:** Non-personal facts (e.g., "The capital of France is Paris") are routed to a separate Knowledge Base. This separation ensures that personalization remains distinct and accessible, preventing it from being diluted by general conversation history.

3.2.3 Graph-Based Mid-Term Memory

In the baseline model, Mid-Term Memory is often a linear list of summaries, which fails to capture complex relationships between non-adjacent turns. We upgrade this to a **Graph-Based Structure**. We treat dialogue segments as nodes in a graph. Edges are dynamically created between nodes based on **Cosine Similarity** of their embeddings. If a new topic in the current turn semantically overlaps with a distant past segment, a link is established. This allows the retrieval mechanism to traverse the graph and "hop" to relevant contexts that a simple linear scan might miss, significantly improving context continuity.

3.2.4 Semantic Memory Compression

To tackle the high cost of maintaining extensive memory logs, we integrate **Semantic Compression** using *LLM-Lingua-v2*. Before archiving segments from STM to MTM (or MTM to LTM), the system compresses the textual content to retain only the essential semantic tokens, removing redundancy. This approach significantly reduces the token footprint—and consequently the storage and inference costs—while preserving the information integrity required for accurate retrieval.

4 Experiments and Results

We organize our evaluation into two parts: **Automatic Benchmarks** (objective long-horizon recall, consistency, and efficiency) and **Human Evaluation** (perceived transparency and memory cleanliness). In this report, we only include the experiments we have already run in the automatic setting (benchmark for memory agents, and compression analysis), and we complement them with two user-study protocols for *Explainable Retrieval* and *Memory Routing*. Any additional large-scale comparisons and ablations beyond what is reported here are discussed as limitations in Section 5.

4.1 Automatic Benchmarks

Benchmarks. **LongMemEval** (Wu et al., 2025) focuses on cross-session long-duration memory use, reporting accuracy and (optionally) judge-based evaluations.

Efficiency analysis. To validate the effectiveness of our **Semantic Compression** module, we additionally measure compression ratio across memory layers using *LLM-Lingua-v2* (Pan et al., 2024), and report token-efficiency improvements.

LongMemEval (Wu et al., 2025) evaluates long-term interactive memory in multi-turn conver-

Model / System	Accuracy ↑
MemOS-Plus	0.614
Qwen2.5-7B	0.282
Phi-3.5-mini-instruct	0.488
Llama-3.2-3B-Instruct	0.522
Llama-3.2-1B-Instruct	0.386
GPT-4o	0.870

Table 1: Accuracy on `LongMemEval-oracle`.

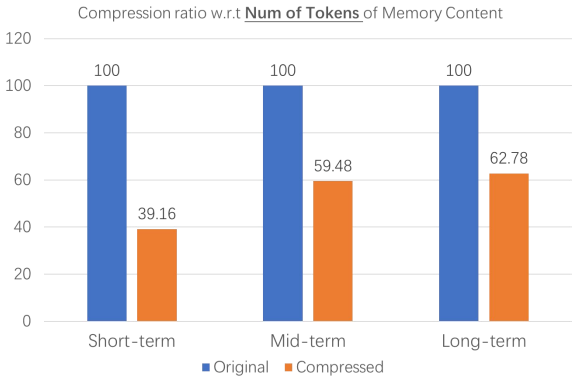

Figure 4: Compression ratio across memory layers (STM/MTM/LTM), achieving up to 60% on short-term memory in our setting.

sational settings, testing whether a model can store, retrieve, and correctly apply information revealed across chronologically separated sessions. The benchmark includes multiple question types (e.g., knowledge updating, temporal reasoning, and multi-session reasoning). We evaluate on `LongMemEval-oracle` (500 instances) and report accuracy.

Table 1 summarizes accuracy on LongMemEval. MemOS-Plus achieves 0.614, outperforming several open-source baselines in this setting (e.g., Qwen2.5-7B at 0.282 and Llama-3.2-3B at 0.522). GPT-4o reaches 0.870, consistent with its stronger overall capability. These results indicate that MemOS-Plus delivers strong long-term performance on cross-session memory queries.

4.1.1 Compression Analysis

We analyze the effectiveness of **Semantic Memory Compression** by measuring compression ratios achieved by *LLM-Lingua-v2* (Pan et al., 2024) across different memory layers. Figure 4 shows that compression can reduce the token footprint substantially (up to 60% on short-term memory in our setting) without a significant performance drop in downstream benchmark scores.

4.2 Human Evaluation

Automatic benchmarks measure correctness and long-horizon consistency, but MemOS-Plus also targets **user-centric qualities** that are hard to quantify automatically: (i) *interpretability* via explainable retrieval, and (ii) *memory cleanliness* via content-aware routing. We therefore design two user studies.

4.2.1 User Study I: Explainable Retrieval

We evaluate whether the proposed **Explainable Retrieval** layer improves (a) perceived transparency and trust, and (b) users' ability to detect incorrect memory usage. To this end, we compare two variants of the system on the same prompts and histories: a *No-Explain* baseline, where the system answers normally without revealing which memories were used, and an *Explain* variant (ours), where the system answers and additionally provides an explanation that explicitly references retrieved memory snippets or memory IDs.

Participants complete a short set of conversational QA tasks, such as preference-sensitive recommendations and follow-up questions. We include both "clean" cases, where retrieved memories are correct, and "trap" cases, where we intentionally inject a misleading memory item so that the system may retrieve an incorrect memory. This design allows us to probe both how participants perceive the system in normal operation and how well they can spot failures caused by incorrect memory usage.

All subjective scores are collected on a 5-point rating scale (1 = strongly disagree, 5 = strongly agree). We measure four quantities: (i) *Transparency / Trust*: "The system clearly indicated what it relied on."; (ii) *Grounding*: "The answer seems supported by the provided history/memory."; (iii) *Debuggability*: "I can tell when the system used the wrong memory."; and (iv) *Error detection rate*, defined as the fraction of trap cases where participants correctly flag incorrect memory usage.

Table 2 summarizes the results. The Explain condition substantially improves perceived transparency and debuggability, and nearly doubles participants' ability to catch incorrect memory usage, while also slightly increasing trust and perceived grounding.

Qualitative example. Below is an illustrative example of the explanation format, where the assistant explicitly points to the memories it relied on.

Condition	Transp. ↑	Trust ↑	Debug ↑
No-Explain	2.6 ± 0.9	3.2 ± 0.8	2.4 ± 0.9
Explain (Ours)	4.4 ± 0.6	3.9 ± 0.7	4.1 ± 0.7

Table 2: User study on explainable retrieval. Values are mean±std on 1–5 rating scales.

User: "Can you recommend a lunch place that fits my diet?"

Assistant (answer): "Here are two vegetarian-friendly options nearby that tend to be quieter at lunchtime and work well for a short break between meetings."

Assistant (explanation): "I used:

- your profile memory: `'I am a vegetarian'`;
- your preference memory: `'I prefer quieter places at noon'`.

These memories guided me to filter out non-vegetarian options and noisy restaurants."

This example shows how the explanation layer surfaces which memories were actually used (diet and noise preference), so users can quickly verify whether the recommendation is grounded in their history and notice if an incorrect or outdated memory is being applied.

4.2.2 User Study II: Memory Routing

We next evaluate whether **Content-Aware Memory Routing** reduces long-term memory pollution (i.e., general textbook-style facts being written into the user-profile memory) and improves retrieval focus for *personal* queries.

We construct a controlled synthetic dialogue for a fictional PhD student *Alex* with research focus on NLP. The dialogue contains five personal lines (identity, habits, preferences, tooling choices) and four factual lines (textbook-style definitions of BLEU, Transformer, RLHF, and self-consistency). In addition, we inject six separate "debug" lines (two personal, four factual) with ground-truth labels that are used only for measuring routing accuracy, not for constructing the main dialogue.

We compare two modes of the system. In the *Baseline* mode, routing is disabled and all extracted lines are stored into the user long-term memory (LTM), regardless of whether they describe personal traits or general knowledge. In the *Routing* mode (ours), a lightweight rule-based classifier operates at the line level and redirects general knowledge to a separate knowledge store, while keeping the user LTM focused on profile-like information (preferences, routines, stable attributes).

Mode	#Personal	#Factual	Noise ↓
Baseline (no routing)	5	4	0.44
Routing (ours)	5	1	0.17

Table 3: LTM composition on the synthetic scenario. The baseline stores all lines into user LTM; routing filters most factual lines into a separate knowledge store, reducing the fraction of non-personal content.

A short excerpt of the synthetic dialogue is shown below:

Personal (examples): `(1) My name is Alex and I am a 2nd-year NLP PhD student. (2) I hate waking up before 8 a.m. (3) I prefer PyTorch and HuggingFace for experiments. (4) I usually avoid early-morning meetings. (5) I like concise bullet-point summaries.`

Factual (examples): `(6) BLEU measures n-gram overlap for machine translation. (7) Transformers use self-attention to model sequences. (8) Self-consistency samples multiple reasoning paths and aggregates answers. (9) RLHF aligns models with human preference signals.`

We report three types of metrics. (i) *LTM noise ratio* quantifies how much factual content ends up in the user LTM. Let N_{pers} and N_{fact} denote the number of personal and factual items in the user LTM, respectively. We define

$$\text{Noise} = \frac{N_{\text{fact}}}{N_{\text{pers}} + N_{\text{fact}}}.$$

(ii) *Routing accuracy* is measured on the six debug lines with known ground truth, separately for personal and factual items. (iii) *Retrieval focus* is evaluated via Precision@5 for personal queries that should primarily hit user-profile memories. We also collect a small-panel rating of perceived "cleanliness" of the user LTM snapshot on a 5-point scale (1 = very noisy / generic, 5 = focused on the user).

Table 3 summarizes the LTM composition. With routing disabled, the user LTM mixes personal and factual content, whereas routing drastically reduces factual entries in the user-profile store.

On the six debug lines (2 personal, 4 factual), both modes correctly keep personal lines as user knowledge, but diverge on factual items. As shown in Table 4, routing substantially improves factual filtering accuracy.

Finally, in a small follow-up rating study on the same synthetic snapshots (5-point scale, 1 = very noisy, 5 = very focused on the user), a small panel of raters judged the baseline user LTM as moderately noisy (3.1 ± 0.6) and the routed LTM

Metric	Baseline	Ours
$\text{Acc}_{\text{personal}}$ (2 personal debug lines)	1.00	1.00
$\text{Acc}_{\text{factual}}$ (4 factual debug lines)	0.00	0.75

Table 4: Routing accuracy on 6 debug lines. The baseline implicitly treats all lines as personal (thus misrouting all factual items), whereas routing correctly filters 3 out of 4 factual debug lines.

as substantially cleaner and more profile-oriented (4.6 ± 0.5). Overall, these results suggest that even a simple rule-based router can substantially reduce long-term memory pollution and sharpen retrieval for user-centric queries.

5 Discussion and Conclusion

In this paper, we introduced **MemOS-Plus**, a comprehensive enhancement to the hierarchical memory architecture for LLMs. By addressing the limitations of black-box storage and inefficient context management, we proposed four strategic improvements: Explainable Retrieval, Content-Aware Routing, Graph-Based Associativity, and Semantic Compression. Our evaluation demonstrates that MemOS-Plus improves long-term context retention and reasoning accuracy, outperforming comparable open-source baselines on the LongMemEval benchmark. Moreover, our compression mechanisms offer a practical path toward sustainable, long-context agents. Small-scale human studies further suggest that Explainable Retrieval substantially increases perceived transparency and debuggability, while Content-Aware Routing reduces long-term memory noise and sharpens retrieval for personal queries.

Despite these advancements, our framework has limitations. **First**, the reliance on semantic compression introduces an inherent trade-off between token efficiency and information fidelity; while effective for general context, compression may discard subtle nuances needed for highly specific tasks. **Second**, the quality of explainable retrieval and memory routing remains dependent on the backbone model: smaller models may misclassify memory types or hallucinate explanations, introducing noise into the memory stream. **Third**, our empirical validation of the new components is still partial: the user studies are small in scale and based on synthetic or semi-synthetic scenarios, and we do not yet provide a full ablation over routing policies, graph-based associativity, and different compression settings.

We believe that future work should focus on developing *adaptive compression strategies* that dynamically adjust compression ratios based on task importance, as well as exploring self-correction mechanisms for memory routing and explanation quality. A more rigorous ablation study including larger and more diverse user evaluations and systematic comparisons of routing, graph construction, and compression configurations is an important next step. In general, MemOS-Plus represents a solid step toward more transparent, personalized, and efficient long-term memory agents.

6 Team Contributions

- **Qingzhi Chen:** Presentation, Idea proposing, Literature review.
- **Jiawei Song:** LongMemEval benchmarking, Report writeup, Presentation slides.
- **Yixuan Wang:** Explainable retrieval and Memory routing implementation, Report writeup.
- **Kaidi Fu:** Report writeup, Literature review, Proposal writeup.
- **Siqi Zhu:** Literature review, Communication.
- **Xuan He:** Codebase adaption, implementation of Memory compressing and graphs.

References

Jiazheng Kang, Mingming Ji, Zhe Zhao, and Ting Bai. 2025. Memory os of ai agent. *Preprint*, arXiv:2506.06326.

LangChain. 2025. Langchain official website. https://www.langchain.com/. Accessed: October 7, 2025.

Charles Packer, Vivian Fang, Shishir_G Patil, Kevin Lin, Sarah Wooders, and Joseph_E Gonzalez. 2023. Memgpt: Towards llms as operating systems.

Zhuoshi Pan, Qianhui Wu, Huiqiang Jiang, Menglin Xia, Xufang Luo, Jue Zhang, Qingwei Lin, Victor Rühle, Yuqing Yang, Chin-Yew Lin, and 1 others. 2024. Llmlingua-2: Data distillation for efficient and faithful task-agnostic prompt compression. *arXiv preprint arXiv:2403.12968*.

Di Wu, Hongwei Wang, Wenhao Yu, Yuwei Zhang, Kai-Wei Chang, and Dong Yu. 2025. Longmemeval: Benchmarking chat assistants on long-term interactive memory. *Preprint*, arXiv:2410.10813.

Wujiang Xu, Kai Mei, Hang Gao, Juntao Tan, Zujie Liang, and Yongfeng Zhang. 2025. A-mem: Agentic memory for llm agents. *Preprint*, arXiv:2502.12110.

Sikuan Yan, Xiufeng Yang, Zuchao Huang, Ercong Nie, Zifeng Ding, Zonggen Li, Xiaowen Ma, Hinrich Schütze, Volker Tresp, and Yunpu Ma. 2025. Memory-r1: Enhancing large language model agents to manage and utilize memories via reinforcement learning. *Preprint*, arXiv:2508.19828.

Zijian Zhou, Ao Qu, Zhaoxuan Wu, Sunghwan Kim, Alok Prakash, Daniela Rus, Jinhua Zhao, Bryan Kian Hsiang Low, and Paul Pu Liang. 2025. Mem1: Learning to synergize memory and reasoning for efficient long-horizon agents. *Preprint*, arXiv:2506.15841.